

Multi-Cohort Simulation Study: Shared vs. Study-Specific Factors (Proposal)

Andrew Walther

2026-06-14

1. Purpose and Background

Prior synthetic validation of the Supervised Bayesian Matrix Factorization (SSBMF) model was carried out only in a **single-cohort** setting. The real application pools several PDAC cohorts that differ both by **platform** (RNA-seq, microarray, proteomics) and by **biology**. In that setting the latent factors fall into two kinds:

- **Shared factors** — gene-expression programs present across *all* studies. These are the cross-cohort biological signal we want to detect and, where they are prognostic, to use for survival prediction.
- **Study-specific factors** — programs private to a *single* study. These absorb platform/batch effects and cohort-private biology that should *not* be mistaken for shared, generalizable signal.

This proposal specifies a simulation that generates data with a **known** shared / study-specific factor structure, so we can test whether the current model **natively** recovers that distinction — the same goal the previously discussed cohort dummy-indicator approach was meant to serve. If the base model already separates shared from study-specific factors, the dummy indicator columns may be unnecessary.

The design incorporates the plan from the last meeting (5/29), with two key elements:

1. **EBMF-grounded ground truth.** Realistic gene programs are taken from an Empirical Bayes Matrix Factorization (`flashier`) fit to the merged real training cohorts. EBMF is also our **unsupervised benchmark** (Section 5).
2. **Shared factors carry survival signal; study-specific factors do not** ($\beta \neq 0$ on shared, $\beta = 0$ on specific). Survival is generated from the shared component only.

2. Notation

Symbol	Meaning	Shape
C	number of cohorts (studies)	scalar
n_c	patients in cohort c ; $n = \sum_c n_c$	scalar
p	number of genes	scalar
K	total number of latent factors	scalar
$\mathcal{S}$	index set of shared factors	$ \mathcal{S} $
$\mathcal{P}_c$	index set of factors specific to cohort c	$ \mathcal{P}_c $
L	patient loadings	$n \times K$
F	gene programs (factors)	$p \times K$
β	log-hazard coefficients	K
Y	observed genomics matrix	$n \times p$
$a_{\text{shared}}, a_{\text{specific}}$	amplitude (signal strength) of shared / specific factors	scalar

Rows of Y (and L) are **stacked by cohort**: rows $1..n_1$ are cohort 1, the next n_2 rows are cohort 2, and so on. Factors partition disjointly: $K = |\mathcal{S}| + \sum_c |\mathcal{P}_c|$. The remaining noise and survival parameters ($\tau_j, \alpha, \lambda_0, U_i$) are defined inline where they first appear in Section 3.

3. Data-Generating Process

3.1 Loadings L ($n \times K$) — block structure encodes specificity

Every loading is drawn non-negative,

$$L_{ik} \sim \text{Exponential}(\text{rate} = 1), \quad \mathbb{E}[L] = 1, \quad \mathbb{E}[L^2] = 2,$$

matching the `point_exponential` prior used by the fitters. Specificity is then imposed by **zeroing out-of-cohort loadings**:

- **Shared factor** $k \in \mathcal{S}$: keep L_{ik} for *all* patients i .
- **Cohort- c -specific factor** $k \in \mathcal{P}_c$: set

$$L_{ik} = 0 \quad \text{for every } i \notin \text{cohort } c.$$

A study-specific factor therefore contributes to Y **only in its own cohort's rows** — this *is* the operational definition of “study-specific.” The resulting L is block-sparse: shared columns are dense, specific columns are zero outside one cohort block.

3.2 Gene Programs / Factors F ($p \times K$)

Each column $F_{\cdot k}$ is a **gene program**: a vector of gene weights, normalized to unit L_2 norm. Two sources are supported:

- **Preferred (EBMF-grounded)**: real gene programs from a `flashier` fit to the merged training cohorts (realistic, sparse). When K exceeds the EBMF rank (the number of programs the `flashier` fit returns), columns are recycled by index.
- **Synthetic fallback** (no real data / no `flashier`): for each factor, pick 5% of genes at random and give them equal weight.

Each column is then scaled by an amplitude — a_{shared} for shared factors, a_{specific} for specific ones — which sets how strongly each factor type contributes signal relative to the noise.

Non-negativity must match the prior. The SSBMF models place a non-negative (`point_exponential`) prior on F , so the ground-truth programs are made non-negative (absolute value) and the EBMF benchmark is run non-negative too. Feeding signed programs to a non-negative-prior model is a prior/data mismatch that artificially depresses recovery — so both are kept non-negative for a fair test.

3.3 Genomics model (Gaussian)

$$Y = LF^\top + E, \quad E_{ij} \sim \mathcal{N}(0, 1/\tau_j), \quad \tau_j \sim \text{Gamma}(\text{shape} = 2, \text{rate} = 2).$$

Each gene j has its own noise precision τ_j , giving noise SD $1/\sqrt{\tau_j}$ ($\mathbb{E}[\tau_j] = 1$), matching the established single-cohort DGP. Y is column-centered before fitting (standard preprocessing). Batch effects are absorbed by the study-specific factors by construction, so no explicit platform offset is added (an optional per-cohort gene-mean offset is available for stress-testing but is off by default).

3.4 Survival model (Weibull proportional hazards) — shared factors only

$$\eta_i = \sum_{k \in \mathcal{S}} L_{ik} \beta_k \quad (\beta_k = 0 \text{ for every specific factor}),$$

$$T_i = \left(\frac{-\log U_i}{\lambda_0 \cdot \exp(\eta_i)} \right)^{1/\alpha}, \quad U_i \sim \text{Unif}(0, 1),$$

where $\alpha = 1.5$ is the Weibull shape and $\lambda_0 = 0.01$ is the baseline scale (controls the overall event rate). The baseline cumulative hazard is $H_0(t) = \lambda_0 t^\alpha$. The default shared coefficients cycle the established pattern $(1.5, -1.2, 0.8, -0.5)$ to length $|\mathcal{S}|$ (truncated if $|\mathcal{S}| < 4$; e.g. $|\mathcal{S}| = 2$ gives $\beta = (1.5, -1.2)$). **Censoring** is calibrated to $\approx 30\%$ by bisection: $\text{censor_base} \sim \text{Exp}(1)$, $\text{time} = \min(\text{event}, \text{censor})$, $\text{status} = \mathbf{1}\{\text{event} \leq \text{censor}\}$.

Anchoring caveat (meeting notes). Because the survival outcome is linked only to shared factors, the supervision “anchors” those factors into the shared component. This is exactly the intended mechanism; the results report discusses whether it produces cleaner shared/specific separation.

4. The Three Scenarios

The scenarios differ only in how the K factors are partitioned into shared vs. specific. For the primary setting $C = 2$, $n_c = 150$, $p = 1000$, $K = 6$:

Scenario	$ \mathcal{S} $ shared ($\beta \neq 0$)	specific per cohort ($\beta = 0$)	Interpretation
All shared	6	0, 0	Studies effectively one cohort
Hybrid	2	2, 2	Both shared and private structure
Nothing shared	0	3, 3	No cross-cohort structure

Nothing-shared is a deliberate false-positive control. With $\mathcal{S} = \emptyset$, $\eta \equiv 0$ and survival carries no factor signal. The correct result is C-index ≈ 0.5 and all $|\hat{\beta}|$ below threshold — the model should *not* fabricate a shared prognostic factor. An optional knob (`specific_prognostic`, default off) can give each cohort a within-cohort prognostic factor if we later want survival signal in this regime.

Worked example — Hybrid, $C = 2$, $n_c = 150$, $p = 1000$, $K = 6$

- **Dimensions:** $n = 300$; Y is 300×1000 , L is 300×6 , F is 1000×6 , β is length 6. Rows 1–150 = cohort 1; 151–300 = cohort 2.
- **Factor labels:** [shared, shared, specific_1, specific_1, specific_2, specific_2].
- **Loadings L :** columns 1–2 dense (all 300 patients). Columns 3–4 zero for rows 151–300 (private to cohort 1). Columns 5–6 zero for rows 1–150 (private to cohort 2). L shows a visible block-sparse pattern.
- **Coefficients:** $\beta = [1.5, -1.2, 0, 0, 0, 0]$ — only the two shared factors drive survival.
- **What a correct model should find:** factors 1–2 with loadings across *both* cohorts *and* nonzero $\hat{\beta}$; factors 3–6 cohort-private with $\hat{\beta} \approx 0$. The unsupervised EBMF benchmark should recover the same six gene programs but cannot say which two are prognostic.

Secondary sensitivity: a 3-cohort hybrid (2 shared + 1 private per cohort), and a sweep of the shared-to-specific amplitude ratio ($a_{\text{shared}}/a_{\text{specific}}$) within the hybrid regime to stress the case where study-specific signal dominates.

5. Model Arms

Each simulated dataset (Y , time, status, cohort_id) is fit by **five arms** — four supervised (LB/YFB $\times$ $\pm$ cohort indicator) plus the unsupervised EBMF benchmark:

Arm	Model	Cohort indicator
YFB_base	YFB ($\eta = (YF)\beta$) — recommended	none
YFB_cohort	YFB	yes
LB_base	LB ($\eta = L\beta$)	none
LB_cohort	LB	yes
EBMF	unsupervised flashier (survival-blind)	—

All supervised arms use a non-negative (`point_exponential`) prior on L and F and a normal prior on β . The cohort indicator, when present, absorbs per-cohort gene-mean differences without entering the survival predictor.

The native-identification test. The headline analysis is the **base** arms (no cohort indicator): can the fitted factors separate into shared vs. study-specific on their own? The $\pm$ indicator comparison then quantifies whether the indicator adds anything.

Why EBMF. EBMF is fit to the same Y but is survival-blind, so it has no β . It is the unsupervised gold standard for the factorization itself, answering two questions:

1. Does the supervised model recover the shared/study-specific structure *as well as* EBMF (i.e., supervision does not cost structural fidelity)?

2. Does supervision *additionally* yield correct β — flagging which factors are prognostic — which EBMF fundamentally cannot?

This is the value-add argument: same structural recovery, plus correct survival attribution.

The EBMF here (fit to each *simulated* Y , scored as a competing method) is distinct from the EBMF in Section 3.2 (fit to *real* data to build ground-truth programs). They do not share a fit.

6. Evaluation Metrics

For each (scenario $\times$ model arm $\times$ seed) we record:

- **Shared-factor recovery** — mean best $|\text{cor}|$ between estimated and true gene programs (F columns, gene space, comparable across LB/YFB/EBMF), over the *true shared* factors.
- **Specific-factor recovery** — same, over the *true study-specific* factors.
- **Specificity-classification accuracy** — label each *estimated* factor shared/specific from its per-cohort loading energy (a factor is called specific if more than 85% of its squared-loading energy concentrates in a single cohort), then compare to the truth. This is the direct test of “does the model natively identify the distinction?”
- **Survival-effect recovery** — $|\hat{\beta}|$ on factors matched to true shared (expect above a small threshold) vs. true specific (expect ≈ 0). Reported as true-positive and **false-positive prognostic rates** (the latter is the anchoring / nothing-shared check).
- **C-index** — orientation-free held-out concordance, $\max(C, 1 - C)$.

Comparisons reported: (i) YFB vs. LB; (ii) base vs. +cohort-dummy (Δ in each metric — “are dummies necessary?”); (iii) supervised vs. unsupervised EBMF.

7. Reference Data-Generating Code

The following is the complete, self-contained DGP (the project implementation wraps this same logic with configuration and seeds). A reader can run it directly given the small `.calibrate` helper.

```
# Bisection calibration of censoring scale to a target censoring fraction.
.calibrate <- function(ev, base, target = 0.30, n_iter = 40) {
  lo <- 1e-3; hi <- max(ev) * 10
  for (i in seq_len(n_iter)) {
    mid <- sqrt(lo * hi)
    if (mean(base * mid < ev) > target) lo <- mid else hi <- mid
  }
  hi
}

generate_multicohort_data <- function(
  C          = 2L,          # number of cohorts
  n_per      = rep(150L, C), # patients per cohort (rows stacked by cohort)
  p          = 1000L,       # genes
  K_shared   = 2L,          # # shared factors      (beta != 0)
  K_specific = rep(2L, C),   # # specific factors per cohort (beta = 0)
  F_templates = NULL,        # p x K_ebmf EBMF programs; NULL -> synthetic sparse
  a_shared   = 1.0,         # amplitude of shared F columns
  a_specific = 1.0,         # amplitude of specific F columns
  active_rate = 0.05,        # synthetic-fallback sparsity
  beta_shared = NULL,        # length K_shared; NULL -> recycle c(1.5,-1.2,0.8,-0.5)
  offset_sd   = 0.0,        # explicit platform offset SD (0 = off)
  shape       = 1.5, scale0 = 0.01, # Weibull-PH baseline (shape = alpha, scale0 = lambda_0)
  target_cens = 0.30,        # target censoring fraction
  seed        = 1L) {
```

```

set.seed(seed)
n <- sum(n_per)
K <- K_shared + sum(K_specific)
cohort_id <- factor(rep(seq_len(C), times = n_per))

# label + owner per column (owner 0 = shared)
labels <- c(rep("shared", K_shared),
            unlist(lapply(seq_len(C), function(cc) rep(paste0("specific_", cc), K_specific[cc]))))
owner <- c(rep(0L, K_shared),
           unlist(lapply(seq_len(C), function(cc) rep(cc, K_specific[cc]))))

# L (n x K): Exponential(1), block-zeroed outside owner cohort
L <- matrix(rexp(n * K, rate = 1), n, K)
for (k in seq_len(K)) if (owner[k] != 0L) L[cohort_id != owner[k], k] <- 0

# F (p x K): EBMF templates (unit-norm) or synthetic sparse (unit-norm)
Fm <- matrix(0, p, K)
if (!is.null(F_templates)) {
  idx <- ((seq_len(K) - 1L) %>% ncol(F_templates)) + 1L
  Fm <- F_templates[, idx, drop = FALSE]
  Fm <- sweep(Fm, 2, sqrt(colSums(Fm^2)) + 1e-10, "/")
} else {
  for (k in seq_len(K)) {
    active <- sample.int(p, max(1L, round(active_rate * p)))
    Fm[active, k] <- 1 / sqrt(length(active))
  }
}
amp <- ifelse(labels == "shared", a_shared, a_specific) # signal-strength knob
Fm <- sweep(Fm, 2, amp, "*")

# per-gene Gaussian noise: tau_j ~ Gamma(2,2)
tau <- rgamma(p, shape = 2, rate = 2)
E <- sweep(matrix(rnorm(n * p), n, p), 2, sqrt(tau), "/")

# optional explicit platform offset (off by default)
offset <- matrix(0, n, p)
if (offset_sd > 0) for (cc in seq_len(C)) {
  ov <- rnorm(p, 0, offset_sd)
  offset[cohort_id == cc, ] <- matrix(ov, sum(cohort_id == cc), p, byrow = TRUE)
}

# assemble + column-center
Y <- L %*% t(Fm) + offset + E
Y <- sweep(Y, 2, colMeans(Y), "-")

# survival from SHARED factors only
if (is.null(beta_shared))
  beta_shared <- if (K_shared > 0) rep_len(c(1.5, -1.2, 0.8, -0.5), K_shared) else numeric(0)
beta <- numeric(K); beta[labels == "shared"] <- beta_shared
eta <- as.vector(L %*% beta) # == 0 in the "nothing shared" scenario

event <- (-log(runif(n)) / (scale0 * exp(eta)))^(1 / shape)
cb <- rexp(n, rate = 1)
cs <- .calibrate(event, cb, target = target_cens)
ctime <- cb * cs
time <- pmin(event, ctime)
status <- as.integer(event <= ctime)

```

```
list(Y = Y, time = time, status = status, cohort_id = cohort_id,
     L_true = L, F_true = Fm, beta_true = beta,
     factor_labels = labels, factor_owner = owner)
}
```

8. Open Questions for Discussion

1. **Specificity definition.** We encode study-specific factors via block-zero loadings (a factor is active in exactly one cohort). Is this the structure you intend, or should specific factors also have *non-overlapping gene programs* per cohort?
2. **Nothing-shared survival.** Default leaves survival null in the nothing-shared scenario (false-positive control). Do you prefer instead a within-cohort prognostic specific factor so each cohort has its own survival signal?
3. **Cohort count.** Primary is $C = 2$ (matches the real training data and the fixed-effects rationale); a $C = 3$ hybrid is included as sensitivity. Is 2 sufficient for the methods claim, or should 3 be primary?
4. **Signal ratio.** Default $a_{\text{shared}} = a_{\text{specific}}$. Should the hybrid scenario stress a regime where study-specific variance *dominates* shared variance (the harder, more realistic case)?

The working implementation and preliminary results are in `results/multi_cohort_sim/` (runner: `run_multicohort_sim.R`; outputs: `outputs/multicohort_sim_results.csv`). A summary results report is available at `docs/reports/multicohort_sim_results.pdf`. Both the implementation and the results report can be updated based on feedback to this proposal.